

Informe Técnico de Sprint

Proyecto: Sistema de Monitoreo Predictivo Industrial IoT | Capa: Hardware Edge & Entorno de Desarrollo

1. Resumen Ejecutivo

El presente informe documenta la transición crítica desde un entorno de desarrollo de prototipos (Arduino IDE) hacia un ecosistema de estándar industrial (PlatformIO sobre Visual Studio Code). Se ha validado exitosamente el código maestro de la capa Edge (ESP32), garantizando la gestión automatizada de dependencias y eliminando falsos positivos en el análisis de sintaxis estática (IntelliSense). El hardware queda formalmente preparado para actuar como el nodo de adquisición de telemetría y control local del proyecto.

2. Marco Teórico y Conceptos Clave

- **Edge Computing (Capa Edge):** Se refiere a la práctica de procesar datos directamente en el lugar donde se generan (en este caso, el microcontrolador ESP32 en la correa transportadora) antes de enviarlos a la nube. Esto reduce la latencia y permite tomar decisiones críticas en tiempo real (Failsafe).
- **CLI (Command Line Interface):** La interfaz de línea de comandos. Representa la forma pura y sin intermediarios gráficos de comunicarse con el núcleo del software (Core). Su uso es vital en ingeniería para evitar errores visuales o limitaciones de interfaces gráficas.
- **Toolchain de Compilación:** Es el conjunto de herramientas de software (compiladores, enlazadores y librerías) que traducen el código C++ legible por humanos a ceros y unos específicos para el chip de Espressif (ESP32).

3. Transición al Estándar Industrial: PlatformIO

Se tomó la decisión estratégica de abandonar Arduino IDE y migrar a **PlatformIO**. Esta transición eleva el estándar del proyecto a nivel profesional por las siguientes razones:

Ventajas de PlatformIO frente a Arduino IDE

- **Gestión de Dependencias (El fin del "Dependency Hell"):** Arduino instala las librerías globalmente en el PC, causando que proyectos distintos choquen si requieren versiones diferentes de la misma librería. PlatformIO utiliza el archivo `platformio.ini` para descargar y aislar las versiones exactas para cada proyecto de forma automática.
- **Trazabilidad en Control de Versiones:** Al definir las librerías en texto (`platformio.ini`), cualquier ingeniero del equipo que clone el repositorio de GitHub tendrá exactamente el mismo entorno, garantizando reproducibilidad.
- **C++ Estándar:** A diferencia de los archivos `.ino` de Arduino que ocultan la estructura real, PlatformIO exige el estándar de C++ usando archivos `.cpp` y la inclusión explícita del framework base mediante `#include <Arduino.h>` en la línea 1.

4. Stack Tecnológico y Librerías del Edge

El archivo de configuración `platformio.ini` instruye la descarga automática de las siguientes dependencias:

Librería	Propósito en el Proyecto Industrial
WiFi.h	Integrada en el Core de Espressif. Permite la conexión a redes inalámbricas.
PubSubClient	Librería ultraligera para implementar el protocolo MQTT, el estándar de la Industria 4.0.
ArduinoJson	Permite la serialización y deserialización de datos para empaquetarlos en formato JSON estandarizado, evitando saturar la memoria RAM.
DHT / Adafruit Unified Sensor	Para la lectura precisa de datos de variables de entorno (Temperatura y Humedad).
ArduinoOTA	(Over The Air). Permite inyectar actualizaciones de firmware al equipo industrial vía WiFi, sin necesidad de conexión por cable USB.
Adafruit GFX / ST7735	Control de la pantalla HMI local para visualización de operarios en terreno.

5. Registro de Troubleshooting (Solución de Errores)

Durante la configuración del entorno, se documentaron y resolvieron obstáculos críticos asociados a la sincronización entre Visual Studio Code y PlatformIO Core:

- **El Falso Positivo de <Arduino.h>:** VS Code marcaba la inclusión con una línea roja a pesar de que el compilador no fallaba. Diagnóstico: El motor IntelliSense (C/C++) no había indexado el diccionario de PlatformIO.
- **Bloqueo de Interfaz Gráfica (0% Download):** La descarga de las herramientas base de C++ se congeló en la interfaz de VS Code debido a problemas de comunicación de red de la extensión visual.
- **Problema de Terminal (PowerShell vs PIO CLI):** El uso de la terminal por defecto de Windows (PS) arrojaba error de comando desconocido, ya que PlatformIO opera en un entorno aislado (Sandbox).

Comandos Maestros (CLI) Utilizados para la Resolución:

```
> pio pkg install // Forzar descarga desde la terminal
> pio run // Compilar e instalar dependencias saltando la UI
> pio init --ide vscode // Regenerar la configuración de VS Code y
limpiar errores visuales
```

Botón Nuclear: Se ejecutó la purga del motor base borrando la carpeta oculta `.platformio` en el directorio de usuario de Windows, obligando al sistema a reinstalar un núcleo limpio y funcional.

6. Estado del Hardware y Actuadores (Pendientes)

La lógica de adquisición de telemetría está cerrada y validada. Sin embargo, en la capa física quedan pendientes críticos antes de cerrar la encapsulación del equipo:

- **Pantalla HMI Local:** Tarea pendiente de diagnóstico de hardware. El instrumento se encuentra "ciego". Falta verificación de emisión de luz (brillo/backlight) y revisión de cableado (SCK a IO18, SDA a IO23).
- **Integración de Actuadores:** El control de los relés industriales se mapeará una vez que el backend Node.js pueda enviar comandos remotos de emergencia para apagar motores.
- **Alimentación:** Infraestructura validada para utilizar fuentes industriales (S-15-5, S-100-12 y módulo ZK-DP60) preparadas para cargas reales de relés y actuadores.

7. Roadmap Inmediato: Próximas Sesiones

Con el Edge encapsulado y el entorno estandarizado, el proyecto asciende a la nube bajo la siguiente hoja de ruta arquitectónica:

1. Fase 2: Backend Node.js (Próximo paso inmediato):

- Inicializar proyecto con `npm init` y TypeScript.
- Instalar y configurar el cliente `mqtt` de Node.js.
- Programar el script `subscriber.ts` para interceptar en tiempo real los JSON que emite el ESP32.

2. Fase 3: Persistencia de Datos:

- Integración del backend con una base de datos Time-Series (como InfluxDB) para guardar los historiales de temperatura, corriente y vibración.

3. Fase 4: API REST:

- Construcción de end-points (`express.js`) para exponer los datos históricos de manera estructurada al frontend.

4. Fase 5: Frontend Industrial (React):

- Desarrollo de un Dashboard con React y Vite para la visualización de KPIs, gráficas y control remoto del sistema.